09 — Frontend Development Guide (Master)

Product: Endpoint IQ **Document:** Frontend Development Guide — how the other 8 documents fit together and the order to build in.

---

1. What the Endpoint IQ Frontend Does

Endpoint IQ automates API testing: it imports an OpenAPI/Swagger spec, logs in as each configured role, runs tests in parallel against every discovered endpoint, and uses an AI engine to analyze failures and produce a developer-friendly report. The **frontend** is the React/TypeScript SPA that lets a developer configure this pipeline (project, spec, roles, credentials), trigger and monitor test runs, and review results, AI analysis, and the final report — without writing a single test by hand. Full detail: **01-Frontend-Requirements.md**.

2. Frontend Architecture (Summary)

Feature-based React + TypeScript SPA, talking only to the API Gateway (REST + WebSocket), with server state cached via a data-fetching library and a small amount of true client-only global state (auth flag, toasts, UI shell). No target-API JWTs or credentials are ever stored in the browser — the backend's HttpOnly cookie model is respected end to end. Full detail: **02-Frontend-Architecture.md**.

3. Repository Structure

```
frontend/
├── src/
│   ├── app/            # providers, router, global layout wiring
│   ├── components/   # shared primitives (Button, Table, Modal, StatusPill, JsonViewer...)
│   ├── features/         # one folder per workflow step (projects, spec-import, endpoints,
│   │                     #   roles, credentials, test-runs, test-results, ai-analysis, reports)
│   ├── pages/           # route-level composition
│   ├── layouts/         # AppLayout, AuthLayout
│   ├── services/        # API client + one module per backend domain
│   ├── hooks/            # shared cross-feature hooks
│   ├── types/            # shared domain types
│   ├── utils/             # formatting, validation, constants
│   ├── stores/            # global client-only state
│   └── routes/            # route config
```

Full rationale: **02-Frontend-Architecture.md §2**.

4. Development Prerequisites

- Node.js + package manager (npm/pnpm/yarn — not specified by source; pick one and standardize).
- Access to a running instance of the API Gateway (or a mocked version) for local development.
- Familiarity with the workflow in **01-Frontend-Requirements.md §4** before touching any feature — every page maps to one step in that flow.

5. Required Environment Variables

Variable	Purpose
`VITE_API_BASE_URL`	API Gateway base URL
`VITE_WS_BASE_URL`	WebSocket base URL for live test-run progress
`VITE_ENV`	Environment name

See **02-Frontend-Architecture.md §13** and **05-Frontend-API-Integration.md §1** for details and open questions on exact naming/versioning.

6. Implementation Order

The phases below follow the product's natural workflow dependency chain (you can't monitor a run before you can start one; you can't start one before roles/credentials exist; etc.), matching the architecture's own step-by-step data flow (§3 of the source document).

Phase 1 — Foundation

- Project scaffold (React + TypeScript, build tool, linting).
- Routing skeleton (all routes from **04-Frontend-Pages-and-Routes.md**, even as stubs).
- `AppLayout` (sidebar + header) per **03-UI-UX-Specification.md §2**.
- Design tokens / shared primitives: `Button`, `Input`, `Table`, `Modal`, `Toast`, `StatusPill`, `Skeleton`, `EmptyState`, `ErrorState`.
- **Docs to reference:** 02-, 03-, 04-.

Phase 2 — Auth & Dashboard

- `/login` (pending confirmation — see Open Decision, 02-… §6) and `/dashboard` / `/projects`.
- Project creation (`/projects/new`).
- **Docs to reference:** 04- (route specs), 05- (auth + project endpoints), 06- (`User`, `Project` types).
- **Backend integration checkpoint:** confirm platform-auth mechanism before building `/login` for real (see Open Decision).

Phase 3 — Spec Import & Endpoint Explorer

- Swagger import flow wired to the **confirmed** `POST /import` endpoint.
- `/projects/:projectId/endpoints` (Endpoint Explorer).
- **Docs to reference:** 05- §18 (Spec Import — Confirmed contract), 06- (`Endpoint` type), 07- (n/a yet).
- **Backend integration checkpoint:** validate import response shape and error cases (bad spec) against §10 Risks in the source architecture.

Phase 4 — Roles & Credentials

- `/projects/:projectId/roles`, `/projects/:projectId/credentials`.
- **Docs to reference:** 04-, 05- (Proposed contracts — confirm with backend), 06- (`Role`, `Credential` types), 08- (write-only credential handling rule).
- **Backend integration checkpoint:** confirm role/credential REST contracts (currently Proposed).

Phase 5 — Test Execution & Progress Monitoring

- `/projects/:projectId/test-runs`, `/projects/:projectId/test-runs/:runId`.
- WebSocket (or polling fallback) live progress hook.
- **Docs to reference:** 04-, 05- §16 (real-time progress — Proposed message schema), 06- (`TestRun` type).
- **Backend integration checkpoint:** confirm WS event schema; confirm graceful polling fallback works if WS is unavailable.
- **Testing checkpoint:** verify a run started in the UI actually reflects true backend job progress, not a client-side simulation.

Phase 6 — Test Results & Failed API Details

- `/projects/:projectId/results`, endpoint result table, `FailedEndpointDetail` (Request/Response/Headers/Diff tabs).
- **Docs to reference:** 07- (full spec for this phase), 05- (results endpoints — Proposed), 06- (`TestResult` type).
- **Testing checkpoint:** validate the UI correctly distinguishes a target-API auth failure (test data) from a frontend/platform auth failure (05-… §9) — this is a common integration bug.

Phase 7 — AI Analysis UI & Developer Recommendations

- `AIAnalysisCard`, Root Cause / Recommended Fix / Developer Fix Prompt blocks, run-level Missing Coverage and Security Findings sections.
- **Docs to reference:** 07- §16-21, 06- (`AIAnalysis`, `SecurityFinding` types — Proposed shape).
- **Backend integration checkpoint:** confirm whether AI analysis is stored per-result, per-run, or both (Open Decision in 06-…).
- Ensure every AI-generated block is visually tagged and never presented as an automated authority (product-behavior requirement, 01-… and 03-… §1).

Phase 8 — Report UI & Download

- `/projects/:projectId/reports/:reportId`, report summary reusing Results/AI components, download action.
- **Docs to reference:** 07- §22, 05- §14 (file download handling — Open Decision on signed URL vs. proxied download).
- **Backend integration checkpoint:** confirm `file_url` delivery mechanism before finalizing the download implementation.

Phase 9 — Testing, Error Handling, Performance, Polish

- Full error-state coverage (all pages) per **03-UI-UX-Specification.md §25** and **05-… §8**.
- Playwright e2e covering the full happy path: create project → import → configure roles/credentials → start run → view results → view AI analysis → download report.
- Pagination/virtualization for large endpoint/result sets if not already addressed (05-… §15, Open Decision).
- Accessibility pass (03-… §28, 08-… §15).
- **Docs to reference:** 08- (all sections), 01-… §10 (Acceptance Criteria — use as the final MVP checklist).

7. Dependencies Between Frontend Modules

```
flowchart TD
    P[Projects] --> SI[Spec Import]
    SI --> EP[Endpoints]
    EP --> R[Roles]
    R --> C[Credentials]
    C --> TR[Test Runs]
    TR --> RES[Results]
    RES --> AI[AI Analysis]
    RES --> REP[Report]
    AI --> REP
```

A feature should not be built before its upstream dependency has at least a stable data shape (even if mocked) — e.g. don't build Test Results against a guessed `TestResult` shape once the real Proposed contract in 05-… has been confirmed with backend.

8. Backend Integration Checkpoints (Consolidated)

Checkpoint	Relevant Open Decision(s)
Platform authentication mechanism	02-… §6, 04-… `/login`
Role/credential REST contracts	05-… §18
WebSocket event schema for run progress	05-… §16
AI analysis storage granularity (per-result vs per-run)	06-… §1 (AIAnalysis)
Report file delivery (signed URL vs. proxied)	05-… §14
Pagination scheme	05-… §15
Run cancellation support	01-… §9, 04-… test-run detail

Every item above must be confirmed with the backend team before the corresponding frontend phase is considered done, not just "in progress."

9. Testing Checkpoints (Consolidated)

- After Phase 3: import a real (or representative sample) OpenAPI spec and confirm the endpoint list renders correctly, including error handling for a malformed spec.
- After Phase 5: confirm live progress reflects real backend state under a genuinely long-running test run, not just a fast local mock.
- After Phase 6: confirm target-API auth failures render as results, not as frontend auth errors (critical distinction, 05-… §9).
- After Phase 8: download a real generated report end-to-end.
- Before release: run the full Playwright happy-path suite and hit the MVP Definition of Done below.

10. MVP Definition of Done

The frontend MVP is complete when every item in **01-Frontend-Requirements.md §10 (Acceptance Criteria)** is satisfied, specifically:

- ☐ Full workflow (project → report) is achievable through the UI alone.
- ☐ Every route in **04-Frontend-Pages-and-Routes.md** is implemented and matches its page spec (purpose, actions, states).

- ☐ All **Confirmed** API contracts in **05-Frontend-API-Integration.md** are integrated; all **Proposed** contracts used have been confirmed with backend (no frontend code shipped against an unconfirmed guess).
- ☐ All domain types in **06-Frontend-State-and-Data-Models.md** match the real backend response shapes (adjusted from Proposed to Confirmed as integration proceeds).
- ☐ Test Results and AI Analysis UI (**07-…**) fully answers the five core developer questions (What failed / Why / Probable cause / How to fix / What to check).
- ☐ All 08-Frontend-Development-Guidelines.md rules are followed — no localStorage tokens, no duplicated components, error/loading states everywhere.
- ☐ Accessibility and responsive baseline (down to tablet) met per **03-UI-UX-Specification.md**.

Document Map

#	Document	Use it when…
01	Frontend-Requirements	Defining scope, MVP boundaries, acceptance criteria
02	Frontend-Architecture	Setting up the project structure, state management, auth handling
03	UI-UX-Specification	Building any visual component or defining visual states
04	Frontend-Pages-and-Routes	Implementing any specific page/route
05	Frontend-API-Integration	Wiring any API call, WS connection, or file download
06	Frontend-State-and-Data-Models	Defining or consuming any TypeScript type
07	Test-Results-and-AI-Report-UI	Building the results/failed-detail/AI-analysis/report UI specifically
08	Frontend-Development-Guidelines	Day-to-day coding conventions and code review
09	Frontend-Development-Guide (this doc)	Planning the build order and tracking integration/testing checkpoints

This document set is intended to live at `docs/frontend/` in the Endpoint IQ repository and be read alongside the *AI-Driven API Testing Platform — Executive Summary & Architecture Guide* (v1.0), which remains the authoritative source for anything not covered here.